
LAPORAN TUGAS BESAR

Implementasi Algoritma Greedy pada Robocode Tank Royale



Disusun oleh:

Angken Muhatun Nufus (124140065) Kevin

Savero (124140143)

Winda Yanti Nainggolan (124140220)

INSTITUT TEKNOLOGI SUMATERA

Program Studi Teknik Informatika IF2211

- Strategi Algoritma

Semester Genap [2026]

DAFTAR ISI

DAFTAR ISI	1
BAB 1: DESKRIPSI TUGAS	2
1.1 Latar Belakang	3
1.2 Tujuan Tugas	3
1.3 Batasan Masalah	4
1.4 Sistematika Laporan	4
BAB 2: LANDASAN TEORI	4
2.1 Algoritma Greedy	5
2.1.1 Pengertian Algoritma Greedy	5
2.1.2 Komponen Algoritma Greedy	5
2.1.3 Kelebihan dan Kekurangan Algoritma Greedy	5
2.2 Robocode Tank Royale	6
2.2.1 Pengenalan Robocode Tank Royale	6
2.2.2 Cara Kerja Program Bot	6
2.2.3 Implementasi Algoritma Greedy pada Bot	6
2.2.4 Cara Menjalankan Bot	7
BAB 3: APLIKASI STRATEGI GREEDY	7
3.1 Mapping Persoalan Robocode Tank Royale ke Elemen Greedy	8
3.2 Eksplorasi 4 Alternatif Solusi Greedy	8
3.2.1 Alternatif 1: Greedy by Distance	8

3.2.2 Alternatif 2: Greedy by Enemy Energy	9
3.2.3 Alternatif 3: Greedy by Hit Probability	9
3.2.4 Alternatif 4: Greedy Defensive Movement	9
3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi	10
3.4 Strategi Greedy yang Dipilih.....	10
BAB 4: IMPLEMENTASI DAN PENGUJIAN	11
4.1 Implementasi Bot Utama Yang Digunakan	12
4.1.1 Pseudocode Bot Utama (Bokir)	12
4.2.2 Pseudocode Bot Alternatif 2 (Jarot)	14
.....	14
4.2.3 Pseudocode Bot Alternatif 3 (Marlong)	15
.....	16
4.2.4 Pseudocode Bot Alternatif 4 (Garit).....	17
4.2 Penjelasan Struktur Data, Fungsi, dan Prosedur Bot Terpilih	17
4.2.1 Struktur Data yang Digunakan	17
4.2.2 Fungsi-Fungsi Utama	17
4.2.3 Prosedur Event Handler	17
4.3 Pengujian	17
4.3.1 Pengujian 1: Kondisi Normal pada Arena Standar	17
4.3.2 Pengujian 2: Melawan Bot Agresif	17
4.3.3 Pengujian 3: [Skenario khusus 2: misal posisi awal terjepit atau bot defensif]	17
4.4 Analisis Hasil Pengujian	17
BAB 5: KESIMPULAN DAN SARAN	17
5.1 Kesimpulan	17

5.2 Saran	17
LAMPIRAN.....	17
Lampiran A: Tautan Repository GitHub	17
Lampiran B: Tautan Video Demo (Opsional)	17
DAFTAR PUSTAKA	17

BAB 1: DESKRIPSI TUGAS

1.1 Latar Belakang

Perkembangan teknologi di bidang kecerdasan buatan membuat banyak permainan komputer mulai memanfaatkan algoritma untuk mengatur perilaku karakter atau agen otomatis. Salah satu algoritma yang cukup sering digunakan adalah algoritma greedy. Algoritma ini bekerja dengan cara memilih keputusan terbaik pada setiap langkah berdasarkan kondisi yang sedang terjadi saat itu.

Pada tugas besar mata kuliah Strategi Algoritma ini, algoritma greedy diterapkan pada permainan Robocode Tank Royale. Robocode Tank Royale merupakan permainan pertarungan tank virtual berbasis pemrograman, di mana pemain harus membuat bot yang dapat bergerak, menyerang, bertahan, dan mengambil keputusan secara otomatis selama pertandingan berlangsung.

Setiap bot harus memiliki strategi agar dapat memperoleh skor setinggi mungkin. Skor dalam permainan diperoleh dari beberapa aspek, seperti damage yang diberikan kepada musuh, kemampuan bertahan hidup, dan keberhasilan mengalahkan lawan. Oleh karena itu, algoritma greedy dipilih karena mampu mengambil keputusan dengan cepat dan cocok digunakan pada permainan yang berjalan secara real-time.

Dalam tugas besar ini dibuat empat strategi greedy yang berbeda. Masing-masing strategi menggunakan heuristic yang berbeda dalam menentukan target maupun tindakan bot. Selanjutnya, seluruh strategi dibandingkan untuk menentukan strategi terbaik yang akan digunakan sebagai bot utama.

Bot utama yang digunakan dalam tugas besar ini adalah bot Bokir. Bot Bokir menggunakan strategi greedy by Distance yaitu bot akan memprioritaskan musuh yang paling dekat untuk dikejar dan ditabrak. Strategi ini dipilih karena musuh yang berada pada jarak dekat lebih mudah diserang dan memberikan peluang lebih besar untuk memperoleh Ram Damage dan Ram Damage Bonus. Selain itu, bot juga menggunakan tembakan dengan power tinggi ketika berada sangat dekat dengan musuh untuk menghasilkan combo damage yang lebih besar.

Melalui tugas besar ini diharapkan mahasiswa dapat memahami penerapan algoritma greedy dalam menyelesaikan permasalahan nyata, khususnya pada pengembangan bot permainan otomatis.

1.2 Tujuan Tugas

Tujuan dari tugas besar ini adalah:

- Memahami konsep dasar algoritma greedy.
- Mengimplementasikan algoritma greedy pada permainan Robocode Tank Royale.
- Membuat beberapa strategi greedy dengan heuristic yang berbeda.
- Menganalisis efektivitas strategi greedy dalam pertandingan.
- Mengembangkan bot yang mampu memperoleh skor tinggi dalam permainan.

1.3 Batasan Masalah

Batasan masalah dalam tugas besar ini adalah sebagai berikut:

- Bot dibuat menggunakan bahasa pemrograman C# (.NET).
- Permainan menggunakan game engine Robocode Tank Royale yang telah dimodifikasi oleh asisten.
- Strategi yang digunakan hanya berbasis algoritma greedy.
- Pengujian dilakukan pada mode pertandingan 1 vs 1.
- Bot hanya memanfaatkan informasi yang diperoleh dari radar dan event permainan.
- Setiap strategi greedy harus memiliki heuristic yang berbeda.

1.4 Sistematika Laporan

Laporan ini disusun dengan sistematika sebagai berikut:

1. Bab 1 membahas deskripsi tugas.
2. Bab 2 membahas landasan teori.
3. Bab 3 membahas aplikasi strategi greedy.
4. Bab 4 membahas implementasi dan pengujian.
5. Bab 5 membahas kesimpulan dan saran.

BAB 2: LANDASAN TEORI

2.1 Algoritma Greedy

2.1.1 Pengertian Algoritma Greedy

Algoritma greedy merupakan salah satu metode penyelesaian masalah yang dilakukan dengan cara memilih keputusan terbaik pada setiap langkah berdasarkan kondisi yang sedang dihadapi saat itu. Pendekatan greedy tidak mempertimbangkan seluruh kemungkinan yang akan terjadi di masa depan, tetapi hanya fokus pada pilihan yang dianggap paling menguntungkan pada langkah sekarang.

Tujuan utama dari algoritma greedy adalah memperoleh solusi yang optimal atau mendekati optimal dengan proses yang lebih cepat dan sederhana dibandingkan metode lainnya. Karena proses pengambilan keputusan dilakukan secara langsung pada setiap langkah, algoritma ini cocok digunakan pada sistem yang membutuhkan respon cepat, salah satunya pada permainan Robocode Tank Royale.

Pada permainan Robocode Tank Royale, bot harus mampu mengambil keputusan dalam waktu yang sangat singkat pada setiap turn. Oleh karena itu, penggunaan algoritma greedy dinilai sesuai karena bot dapat langsung menentukan tindakan terbaik seperti memilih target, menentukan arah gerakan, maupun menentukan waktu menyerang berdasarkan kondisi arena saat itu.

2.1.2 Komponen Algoritma Greedy

Algoritma greedy memiliki komponen-komponen utama berikut:

1. Himpunan kandidat:
Sekumpulan elemen yang dapat dipilih untuk membentuk solusi.
2. Himpunan solusi:
Kumpulan kandidat yang telah dipilih menjadi solusi sementara atau solusi akhir.
3. Fungsi seleksi:
Fungsi yang menentukan kandidat terbaik yang dipilih pada setiap langkah.
4. Fungsi kelayakan:
Fungsi yang memeriksa apakah kandidat yang dipilih memenuhi syarat untuk dimasukkan ke solusi.
5. Fungsi objektif:
Fungsi yang digunakan untuk mengukur kualitas solusi, misalnya memaksimalkan damage atau meminimalkan risiko.

2.1.3 Kelebihan dan Kekurangan Algoritma Greedy

Algoritma greedy memiliki beberapa kelebihan, di antaranya mudah dipahami dan diimplementasikan. Selain itu, proses eksekusinya relatif cepat karena hanya memilih solusi terbaik pada setiap langkah tanpa harus memeriksa seluruh kemungkinan solusi yang ada. Hal ini membuat algoritma greedy cocok digunakan pada sistem real-time seperti Robocode Tank Royale yang memiliki batas waktu pada setiap turn.

Meskipun demikian, algoritma greedy juga memiliki kekurangan. Algoritma ini tidak selalu menghasilkan solusi global yang paling optimal karena keputusan yang diambil hanya berdasarkan kondisi saat itu. Pada beberapa kondisi tertentu, pilihan yang terlihat paling baik di awal belum tentu menghasilkan hasil terbaik pada akhir pertandingan. Selain itu, performa algoritma greedy sangat bergantung pada heuristic yang digunakan. Jika heuristic yang dipilih kurang tepat, maka performa bot juga dapat menurun.

2.2 Robocode Tank Royale

2.2.1 Pengenalan Robocode Tank Royale

Robocode Tank Royale adalah permainan simulasi pertarungan tank berbasis pemrograman. Pada permainan ini, pemain membuat bot otomatis menggunakan bahasa pemrograman tertentu untuk bertarung di dalam arena.

Setiap bot memiliki kemampuan untuk:

- Bergerak di arena.
- Memutar radar dan meriam.

- Mendeteksi musuh.
- Menembak musuh.
- Menghindari serangan lawan.

Pemenang pertandingan ditentukan berdasarkan skor dan kemampuan bertahan hidup bot.

2.2.2 Cara Kerja Program Bot

Bot bekerja berdasarkan loop utama yang dijalankan terus-menerus selama pertandingan berlangsung. Pada setiap turn, bot akan:

1. Memindai arena menggunakan radar.
2. Mengumpulkan informasi musuh.
3. Memilih aksi berdasarkan strategi tertentu.
4. Menjalankan aksi seperti bergerak atau menembak.

Robocode menyediakan berbagai event handler seperti:

- OnScannedBot
- OnHitWall
- OnHitBot
- OnBotDeath

Event tersebut digunakan agar bot dapat bereaksi terhadap kondisi permainan secara real-time.

2.2.3 Implementasi Algoritma Greedy pada Bot

Implementasi greedy dilakukan dengan memilih aksi terbaik pada setiap turn berdasarkan heuristik tertentu. Contohnya:

- Memilih musuh terdekat.
- Memilih musuh dengan energi terendah.
- Memilih target yang paling mudah ditembak.

Bot tidak menghitung semua kemungkinan masa depan, melainkan langsung mengambil keputusan yang dianggap paling menguntungkan saat itu.

2.2.4 Cara Menjalankan Bot

Langkah menjalankan bot pada Robocode Tank Royale:

1. Install Java Development Kit (JDK).
2. Install .NET SDK dan Robocode Tank Royale.
3. Compile file bot menggunakan Visual Studio atau terminal.
4. Jalankan server Robocode Tank Royale.
5. Jalankan bot menggunakan command: dotnet run
6. Pilih bot dan mulai pertandingan.

BAB 3: APLIKASI STRATEGI GREEDY

3.1 Mapping Persoalan Robocode Tank Royale ke Elemen Greedy

Pada permainan Robocode Tank Royale, bot harus mampu mengambil keputusan dengan cepat di setiap turn agar dapat memperoleh skor setinggi mungkin. Permasalahan ini dapat dimodelkan ke dalam elemen-elemen algoritma greedy karena bot harus selalu memilih aksi yang dianggap paling menguntungkan berdasarkan kondisi saat itu juga.

Elemen Greedy	Konteks Robocode Tank Royale
Himpunan Kandidat	Semua musuh yang berhasil dideteksi radar bot selama pertandingan berlangsung.
Himpunan Solusi	Kumpulan aksi yang dilakukan bot seperti bergerak, mengejar musuh, menghindari, menembak, dan menabrak lawan.
Fungsi Solusi	Kondisi ketika bot berhasil memperoleh skor tinggi atau memenangkan pertandingan dengan mengalahkan lawan.
Fungsi Seleksi	Proses memilih target atau aksi terbaik berdasarkan heuristic tertentu, misalnya memilih musuh terdekat atau musuh dengan energi terendah.
Fungsi Kelayakan	Memastikan aksi yang dipilih dapat dilakukan, misalnya posisi musuh masih dalam jangkauan radar atau meriam tidak dalam kondisi panas.
Fungsi Objektif	Memaksimalkan total skor pertandingan melalui bullet damage, survival score, ram damage, dan bonus kemenangan.

Pada implementasinya, bot akan terus melakukan scanning terhadap arena. Setelah musuh ditemukan, bot akan menentukan tindakan terbaik berdasarkan strategi greedy yang digunakan. Keputusan tersebut dilakukan secara berulang pada setiap turn sehingga bot dapat terus beradaptasi terhadap kondisi pertandingan.

3.2 Eksplorasi 4 Alternatif Solusi Greedy

Dalam tugas besar ini dibuat empat alternatif strategi greedy yang berbeda. Masing-masing strategi menggunakan heuristic yang berbeda dalam menentukan target maupun tindakan bot selama pertandingan berlangsung. Tujuan dari eksplorasi ini adalah mencari strategi yang paling efektif untuk memperoleh skor tertinggi pada permainan Robocode Tank Royale.

3.2.1 Alternatif 1: Greedy by Distance

Strategi ini diterapkan pada bot Bokir. Bot akan selalu memilih musuh yang memiliki jarak paling dekat lalu langsung bergerak mengejar target tersebut untuk melakukan tabrakan atau serangan jarak dekat.

Strategi ini termasuk algoritma greedy karena pada setiap putaran bot selalu mengambil keputusan lokal terbaik, yaitu memilih musuh terdekat tanpa mempertimbangkan kondisi jangka panjang.

Bokir memanfaatkan ram damage dan ram damage bonus sebagai sumber skor utama. Ketika berhasil mendekati lawan, bot akan menembakkan peluru dengan power maksimum untuk menghasilkan combo damage.

Heuristic yang digunakan:

- Memilih musuh dengan jarak paling dekat.
- Semakin dekat musuh, semakin tinggi prioritas target.
- Fokus pada tabrakan langsung untuk memaksimalkan damage.

3.2.2 Alternatif 2: Greedy by Enemy Energy

Strategi ini digunakan pada bot Marlong. Bot memprioritaskan musuh yang memiliki energi paling rendah agar lawan dapat dieliminasi lebih cepat.

Bot akan terus memantau energi seluruh musuh lalu memilih target dengan HP terendah sebagai prioritas utama. Strategi ini bertujuan memperoleh bullet damage bonus dan survival score dengan lebih efektif.

Strategi ini termasuk greedy karena bot selalu memilih target yang paling mudah dikalahkan pada kondisi saat itu.

Heuristic yang digunakan:

- Memilih musuh dengan energi paling rendah.
- Memprioritaskan lawan yang hampir hancur.
- Mengurangi jumlah ancaman di arena secepat mungkin.

3.2.3 Alternatif 3: Greedy by Hit Probability

Strategi ini diterapkan pada bot Jarot. Bot akan memilih target dengan peluang terkena tembakan paling tinggi berdasarkan kecepatan gerak lawan.

Jarot menggunakan prediksi arah gerakan musuh untuk menentukan sudut tembak terbaik. Musuh yang bergerak lebih lambat akan menjadi prioritas karena peluang peluru mengenai target lebih besar.

Strategi ini termasuk greedy karena bot selalu memilih peluang hit terbaik pada setiap putaran permainan.

Heuristic yang digunakan:

- Memilih musuh dengan probabilitas hit tertinggi.
- Memprediksi arah gerakan lawan sebelum menembak.
- Menggunakan bullet power adaptif berdasarkan jarak target.

3.2.4 Alternatif 4: Greedy Defensive Movement

Strategi ini digunakan pada bot Garit. Fokus utama bot adalah bertahan hidup dengan menghindari serangan lawan sambil tetap mencari kesempatan menyerang.

Garit menggunakan pergerakan menyamping dan perubahan arah mendadak untuk mengurangi peluang terkena peluru. Selain itu, bot tetap memilih target tertentu untuk diserang ketika kondisi memungkinkan.

Strategi ini termasuk greedy karena bot selalu memilih gerakan yang dianggap paling aman pada kondisi saat itu tanpa melakukan perencanaan jangka panjang.

Heuristic yang digunakan:

- Memilih gerakan dengan risiko terkena peluru paling kecil.
- Mengubah arah ketika musuh menembak.
- Menyerang target ketika posisi sudah aman.

3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi

Keempat strategi greedy memiliki karakteristik yang berbeda-beda. Perbandingan dilakukan berdasarkan kompleksitas algoritma, efektivitas saat pertandingan, serta kelebihan dan kekurangan masing-masing strategi.

Alternatif	Kompleksitas	Efektivitas	Kelebihan	Kekurangan
Greedy by Distance	$O(n)$	Tinggi	Agresif, sederhana, cocok untuk ramming	Rentan terkena serangan balik
Greedy by Enemy Energy	$O(n)$	Sedang	Cepat mengeliminasi lawan lemah	Sering mengabaikan posisi musuh
Greedy by Angle Prediction	$O(n)$	Tinggi	Cepat mengeliminasi lawan lemah	Perhitungan lebih kompleks
Greedy by Safe Movement	$O(n)$	Sedang	Survival lebih baik	Kurang agresif menghasilkan damage

Keterangan:

- n merupakan jumlah musuh yang berhasil dipindai radar.
- Semua strategi menggunakan pendekatan greedy karena keputusan diambil berdasarkan kondisi terbaik pada saat itu tanpa mempertimbangkan seluruh kemungkinan di masa depan.

Dari hasil analisis, strategi Greedy by Distance dipilih sebagai strategi utama pada bot Bokir karena paling sesuai dengan gaya permainan agresif yang digunakan. Strategi ini mampu menghasilkan damage dan ram damage lebih cepat dibanding alternatif lainnya.

3.4 Strategi Greedy yang Dipilih

Berdasarkan hasil analisis terhadap empat alternatif strategi greedy yang telah dibuat, strategi yang dipilih untuk diimplementasikan pada bot utama adalah Greedy by Distance. Strategi ini dipilih karena memiliki performa yang paling sesuai dengan karakter permainan bot Bokir yang agresif dan berfokus pada pertarungan jarak dekat.

Pada strategi ini, bot akan selalu memilih musuh dengan jarak paling dekat sebagai target utama. Heuristic tersebut dinilai efektif karena musuh yang berada pada jarak dekat lebih mudah dikejar untuk melakukan tabrakan langsung atau ram attack. Selain itu, jarak yang dekat juga membuat peluang serangan mengenai target menjadi lebih besar dibandingkan serangan dari jarak jauh.

Strategi Greedy by Distance memiliki proses pengambilan keputusan yang sederhana dan cepat. Hal ini penting dalam permainan Robocode Tank Royale karena setiap bot harus mampu merespons kondisi pertandingan secara real-time. Dengan mekanisme seleksi target yang sederhana, bot dapat bergerak dan menyerang tanpa membutuhkan proses perhitungan yang kompleks.

Dibandingkan alternatif strategi lainnya, pendekatan ini lebih cocok digunakan pada bot Bokir karena mampu menghasilkan ram damage secara konsisten. Bot dapat terus memberikan tekanan kepada lawan dengan bergerak mendekati target sambil melakukan serangan secara terus-menerus. Ketika berhasil berada sangat dekat dengan lawan, bot juga akan menembakkan peluru dengan power maksimum untuk meningkatkan total damage yang dihasilkan.

Selain itu, strategi ini cukup efektif digunakan pada pertandingan 1 vs 1 karena fokus serangan hanya tertuju pada satu target terdekat. Dengan pendekatan tersebut, peluang bot untuk mendominasi duel menjadi lebih besar karena lawan sulit menjaga jarak aman dari Bokir.

Oleh karena itu, strategi Greedy by Distance dipilih sebagai strategi utama pada bot Bokir karena mampu memberikan keseimbangan antara kesederhanaan algoritma, kecepatan pengambilan keputusan, dan efektivitas dalam menghasilkan damage selama pertandingan berlangsung.

BAB 4: IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi Bot Utama Yang Digunakan

4.1.1 Pseudocode Bot Utama (Bokir)

```
C# Bokir.cs
1  using System;
2  using System.Drawing;
3  using System.Collections.Generic;
4  using Robocode.TankRoyale.BotApi;
5  using Robocode.TankRoyale.BotApi.Events;
6
7  public class Bokir : Bot
8  {
9      private readonly Dictionary<int, ScannedBotEvent> _scannedEnemies = new();
10
11     private int _closestEnemyId = -1;
12     private double _closestDistance = double.MaxValue;
13
14     private bool _inRamMode = false;
15
16     static void Main(string[] args) { new Bokir().Start(); }
17
18     Bokir() : base(BotInfo.FromFile("Bokir.json")) { }
19
20     public override void Run()
21     {
22         BodyColor = Color.FromArgb(128, 0, 0);
23         TurretColor = Color.FromArgb(40, 40, 40);
24         RadarColor = Color.Crimson;
25         BulletColor = Color.Red;
26         ScanColor = Color.FromArgb(255, 69, 0);
27
28         SetTurnRadarRight(double.PositiveInfinity);
29
30         while (IsRunning)
31         {
32             SelectClosestEnemy();
33
34             if (_closestEnemyId >= 0 && _scannedEnemies.TryGetValue(_closestEnemyId, out var target))
35             {
36                 ChaseAndRam(target);
37             }
38             else
39             {
40                 SetTurnRight(15);
41                 SetForward(50);
42             }
43
44             Go();
45         }
46     }
```

```

48     private void SelectClosestEnemy()
49     {
50         _closestEnemyId = -1;
51         _closestDistance = double.MaxValue;
52
53         foreach (var kvp in _scannedEnemies)
54         {
55             double dist = DistanceTo(kvp.Value.X, kvp.Value.Y);
56
57             if (dist < _closestDistance)
58             {
59                 _closestDistance = dist;
60                 _closestEnemyId = kvp.Key;
61             }
62         }
63
64         _inRamMode = _closestDistance < 50;
65     }

```

```

67     private void ChaseAndRam(ScannedBotEvent target)
68     {
69         double bearingToEnemy = BearingTo(target.X, target.Y);
70
71         SetTurnLeft(bearingToEnemy);
72         SetForward(double.PositiveInfinity);
73
74         double gunBearing = GunBearingTo(target.X, target.Y);
75         SetTurnGunLeft(gunBearing);
76
77         if (_inRamMode && Math.Abs(gunBearing) < 15)
78         {
79             SetFire(3.0);
80         }
81         else if (_closestDistance < 150 && Math.Abs(gunBearing) < 20)
82         {
83             SetFire(2.0);
84         }
85
86         double radarBearing = RadarBearingTo(target.X, target.Y);
87         SetTurnRadarLeft(radarBearing);
88     }

```



```

90     public override void OnScannedBot(ScannedBotEvent e)
91     {
92         _scannedEnemies[e.ScannedBotId] = e;
93     }
94
95     public override void OnBotDeath(BotDeathEvent e)
96     {
97         _scannedEnemies.Remove(e.VictimId);
98
99         if (_closestEnemyId == e.VictimId)
100         {
101             _closestEnemyId = -1;
102             _closestDistance = double.MaxValue;
103         }
104     }
105
106     public override void OnHitWall(HitWallEvent e)
107     {
108         SetBack(30);
109         SetTurnRight(45);
110     }
111
112     public override void OnHitBot(HitBotEvent e)
113     {
114         SetFire(3.0);
115         SetForward(50);
116     }
117 }
118

```

4.2.2 Pseudocode Bot Alternatif 2 (Jarot)

```

1  using System;
2  using System.Drawing;
3  using System.Collections.Generic;
4  using Robocode.TankRoyale.BotApi;
5  using Robocode.TankRoyale.BotApi.Events;
6
7  public class Jarot : Bot
8  {
9      private readonly Dictionary<int, ScannedBotEvent> _scannedEnemies = new();
10
11      private int _sniperTargetId = -1;
12      private double _targetSpeed = double.MaxValue;
13      private double _targetDistance = 0;
14
15      private int _strafeDirection = 1;
16      private int _turnsSinceStrafe = 0;
17      private const int StrafePeriod = 30;
18
19      static void Main(string[] args) { new Jarot().Start(); }
20
21      Jarot() : base(BotInfo.FromFile("Jarot.json")) { }
22

```

```

23     public override void Run()
24     {
25         BodyColor    = Color.Black;
26         TurretColor  = Color.DarkRed;
27         RadarColor   = Color.LimeGreen;
28         BulletColor  = Color.Red;
29         ScanColor    = Color.Lime;
30
31         SetTurnRadarRight(double.PositiveInfinity);
32
33         while (IsRunning)
34         {
35             _turnsSinceStrafe++;
36
37             SelectHighestHitProbabilityTarget();
38
39             if (_sniperTargetId >= 0 && _scannedEnemies.TryGetValue(_sniperTargetId, out var target))
40             {
41                 ExecuteSniperShot(target);
42             }
43
44             ExecutePerpendicularMovement();
45
46             Go();
47         }
48     }
49

```

```

50     private void SelectHighestHitProbabilityTarget()
51     {
52         _sniperTargetId = -1;
53         _targetSpeed    = double.MaxValue;
54
55         foreach (var kvp in _scannedEnemies)
56         {
57             double speed = Math.Abs(kvp.Value.Speed);
58
59             if (speed < _targetSpeed ||
60                 (Math.Abs(speed - _targetSpeed) < 0.5 &&
61                     DistanceTo(kvp.Value.X, kvp.Value.Y) < _targetDistance))
62             {
63                 _targetSpeed    = speed;
64                 _sniperTargetId = kvp.Key;
65                 _targetDistance = DistanceTo(kvp.Value.X, kvp.Value.Y);
66             }
67         }
68     }
69

```



```

70 private void ExecuteSniperShot(ScannedBotEvent target)
71 {
72     double distance = DistanceTo(target.X, target.Y);
73
74     double bulletPower;
75     if (distance > 400)
76     {
77         bulletPower = 1.0;
78     }
79     else if (distance > 200)
80     {
81         bulletPower = 2.0;
82     }
83     else
84     {
85         bulletPower = 3.0;
86     }
87
88     double bulletSpeed = 20 - 3 * bulletPower;
89     double travelTime = distance / bulletSpeed;
90
91     double futureX = target.X + Math.Sin(DegreesToRadians(target.Direction)) * target.Speed * travelTime;
92     double futureY = target.Y + Math.Cos(DegreesToRadians(target.Direction)) * target.Speed * travelTime;
93
94     double gunBearingToFuturePos = GunBearingTo(futureX, futureY);
95     SetTurnGunLeft(gunBearingToFuturePos);
96

```

```

97     if (Math.Abs(gunBearingToFuturePos) < 5)
98     {
99         SetFire(bulletPower);
100     }
101
102     double radarBearing = RadarBearingTo(target.X, target.Y);
103     if (Math.Abs(radarBearing) > 5)
104     {
105         SetTurnRadarLeft(radarBearing * 1.2);
106     }
107 }
108
109 private void ExecutePerpendicularMovement()
110 {
111     if (_turnsSinceStrafe >= StrafePeriod)
112     {
113         _strafeDirection *= -1;
114         _turnsSinceStrafe = 0;
115     }
116
117     if (_sniperTargetId >= 0 && _scannedEnemies.TryGetValue(_sniperTargetId, out var target))
118     {
119         double bearingToTarget = BearingTo(target.X, target.Y);
120
121         double perpendicularBearing = bearingToTarget + (90 * _strafeDirection);
122         SetTurnLeft(NormalizeRelativeAngle(perpendicularBearing));
123         SetForward(80);
124     }
125     else
126     {
127         SetTurnRight(20);
128         SetForward(50);
129     }

```

```

131     AvoidWallsSniper();
132 }
133
134 private void AvoidWallsSniper()
135 {
136     double margin = 100;
137     if (X < margin || X > (ArenaWidth - margin) ||
138         Y < margin || Y > (ArenaHeight - margin))
139     {
140         double centerBearing = BearingTo(ArenaWidth / 2.0, ArenaHeight / 2.0);
141         SetTurnLeft(centerBearing);
142         SetForward(120);
143     }
144 }
145
146 private static double DegreesToRadians(double degrees)
147     => degrees * Math.PI / 180.0;
148
149 public override void OnScannedBot(ScannedBotEvent e)
150 {
151     _scannedEnemies[e.ScannedBotId] = e;
152 }
153
154 public override void OnBotDeath(BotDeathEvent e)
155 {
156     _scannedEnemies.Remove(e.VictimId);
157     if (_sniperTargetId == e.VictimId)
158         _sniperTargetId = -1;
159 }
160
161 public override void OnHitWall(HitWallEvent e)
162 {
163     SetBack(40);
164     SetTurnRight(90);
165     _strafeDirection *= -1;

```

```

168     public override void OnHitBot(HitBotEvent e)
169     {
170         SetBack(60);
171         SetFire(2.0);
172     }
173 }
174

```

4.2.3 Pseudocode Bot Alternatif 3 (Marlong)

```
1  using System;
2  using System.Drawing;
3  using System.Collections.Generic;
4  using Robocode.TankRoyale.BotApi;
5  using Robocode.TankRoyale.BotApi.Events;
6
7  public class Marlong : Bot
8  {
9      private readonly Dictionary<int, ScannedBotEvent> _scannedEnemies = new();
10     private int _currentTargetId = -1;
11     private int _turnCount = 0;
12
13     static void Main(string[] args) { new Marlong().Start(); }
14
15     Marlong() : base(BotInfo.FromFile("Marlong.json")) { }
16
17     public override void Run()
18     {
19         BodyColor = Color.MidnightBlue;
20         TurretColor = Color.Black;
21         RadarColor = Color.DarkSlateGray;
22         BulletColor = Color.Gold;
23         ScanColor = Color.Cyan;
24
25         SetTurnRadarRight(double.PositiveInfinity);
26
27         while (IsRunning)
28         {
29             _turnCount++;
30
31             SelectGreedyTarget();
32
33             if (_currentTargetId >= 0 && _scannedEnemies.TryGetValue(_currentTargetId, out var target))
34             {
35                 ExecuteGreedyAttack(target);
36             }
37
38             ExecuteSurvivalMovement();
39
40             Go();
41         }
42     }
43
44     private void SelectGreedyTarget()
45     {
46         if (_scannedEnemies.Count == 0)
47         {
48             _currentTargetId = -1;
49             return;
50         }
51
52         int bestId = -1;
53         double lowestEnergy = double.MaxValue;
54     }
```

```

45     {
46         foreach (var kvp in _scannedEnemies)
47         {
48             if (kvp.Value.Energy < lowestEnergy)
49             {
50                 lowestEnergy = kvp.Value.Energy;
51                 bestId       = kvp.Key;
52             }
53         }
54
55         _currentTargetId = bestId;
56     }
57
58     private void ExecuteGreedyAttack(ScannedBotEvent target)
59     {
60         double bearingToTarget = BearingTo(target.X, target.Y);
61         double gunTurnNeeded   = NormalizeRelativeAngle(GunDirection - Direction + bearingToTarget);
62
63         SetTurnGunLeft(gunTurnNeeded);
64
65         double bulletPower;
66         if (target.Energy <= 4.0)
67         {
68             bulletPower = 1.0;
69         }
70         else if (target.Energy <= 16.0)
71         {
72             bulletPower = 2.0;
73         }
74         else
75         {
76             bulletPower = 3.0;
77         }
78
79         if (Math.Abs(GunBearingTo(target.X, target.Y)) < 10)
80         {
81             SetFire(bulletPower);
82         }
83     }
84
85     private void ExecuteSurvivalMovement()
86     {
87         if (_scannedEnemies.Count == 0)
88         {
89             SetTurnRight(10);
90             SetForward(50);
91             return;
92         }
93
94         double avgX = 0, avgY = 0;
95         foreach (var kvp in _scannedEnemies)
96         {
97             avgX += kvp.Value.X;
98             avgY += kvp.Value.Y;
99         }
100         avgX /= _scannedEnemies.Count;
101         avgY /= _scannedEnemies.Count;
102
103         double bearingToCrowd = BearingTo(avgX, avgY);
104         double escapeDirection = NormalizeRelativeAngle(bearingToCrowd + 180);
105
106         SetTurnLeft(escapeDirection);
107         SetForward(100);
108         AvoidWalls();
109     }
110
111     private void AvoidWalls()
112     {
113         double margin = 80;
114         double x = X, y = Y;

```

```

90         SetFire(bulletPower);
91     }
92 }
93
94 private void ExecuteSurvivalMovement()
95 {
96     if (_scannedEnemies.Count == 0)
97     {
98         SetTurnRight(10);
99         SetForward(50);
100         return;
101     }
102
103     double avgX = 0, avgY = 0;
104     foreach (var kvp in _scannedEnemies)
105     {
106         avgX += kvp.Value.X;
107         avgY += kvp.Value.Y;
108     }
109     avgX /= _scannedEnemies.Count;
110     avgY /= _scannedEnemies.Count;
111
112     double bearingToCrowd = BearingTo(avgX, avgY);
113     double escapeDirection = NormalizeRelativeAngle(bearingToCrowd + 180);
114
115     SetTurnLeft(escapeDirection);
116     SetForward(100);
117     AvoidWalls();
118 }
119 private void AvoidWalls()
120 {
121     double margin = 80;
122     double x = X, y = Y;

```



```

124         bool nearWall = x < margin || x > (ArenaWidth - margin) ||
125         | | y < margin || y > (ArenaHeight - margin);
126
127         if (nearWall)
128         {
129             double centerBearing = BearingTo(ArenaWidth / 2.0, ArenaHeight / 2.0);
130             SetTurnLeft(centerBearing);
131             SetForward(150);
132         }
133     }
134     public override void OnScannedBot(ScannedBotEvent e)
135     {
136         _scannedEnemies[e.ScannedBotId] = e;
137     }
138     public override void OnBotDeath(BotDeathEvent e)
139     {
140         _scannedEnemies.Remove(e.VictimId);
141
142         if (_currentTargetId == e.VictimId)
143         {
144             _currentTargetId = -1;
145         }
146     }
147
148     public override void OnHitWall(HitWallEvent e)
149     {
150         SetBack(50);
151         SetTurnRight(90);
152     }
153     public override void OnHitBot(HitBotEvent e)
154     {
155         if (e.IsRammed)
156         {
157             SetFire(3.0);

```

4.2.4 Pseudocode Bot Alternatif 4 (Garit)

```

1  using System;
2  using System.Drawing;
3  using System.Collections.Generic;
4  using Robocode.TankRoyale.BotApi;
5  using Robocode.TankRoyale.BotApi.Events;
6
7  public class Garit : Bot
8  {
9      private double _moveDirection = 1;
10
11      private readonly Dictionary<int, ScannedBotEvent> _detectedEnemies = new();
12      private readonly Dictionary<int, double> _enemyEnergyHistory = new();
13      private readonly Random _rand = new();
14
15      private int _targetBotId = -1;
16
17      static void Main(string[] args) { new Garit().Start(); }
18
19      Garit() : base(BotInfo.FromFile("Garit.json")) { }
20
21      public override void Run()
22      {
23          BodyColor = Color.Black;
24          TurretColor = Color.FromArgb(20, 20, 20);
25          RadarColor = Color.FromArgb(40, 40, 40);
26          BulletColor = Color.White;
27          ScanColor = Color.FromArgb(30, 30, 30);
28
29          SetTurnRadarRight(double.PositiveInfinity);
30
31          while (IsRunning)
32          {
33              SelectWeakestTarget();
34
35              SetForward(100 * _moveDirection);
36

```

```

37         if (_targetBotId == -1)
38         {
39             if (RadarTurnRemaining == 0)
40             {
41                 SetTurnRadarRight(double.PositiveInfinity);
42             }
43             SetTurnRight(15);
44         }
45     }
46     Go();
47 }
48
49
50 private void SelectWeakestTarget()
51 {
52     if (_detectedEnemies.Count == 0) { _targetBotId = -1; return; }
53
54     int bestId = -1;
55     double lowestHP = double.MaxValue;
56
57     foreach (var kvp in _detectedEnemies)
58     {
59         if (kvp.Value.Energy < lowestHP)
60         {
61             lowestHP = kvp.Value.Energy;
62             bestId = kvp.Key;
63         }
64     }
65     _targetBotId = bestId;
66 }
67

```

```

68 public override void OnScannedBot(ScannedBotEvent e)
69 {
70     _detectedEnemies[e.ScannedBotId] = e;
71
72     double distance = DistanceTo(e.X, e.Y);
73     double bearing = BearingTo(e.X, e.Y);
74
75     if (!_enemyEnergyHistory.TryGetValue(e.ScannedBotId, out double previousEnergy))
76     {
77         previousEnergy = 100.0;
78     }
79
80     double energyDrop = previousEnergy - e.Energy;
81     if (energyDrop > 0 && energyDrop <= 3.0)
82     {
83         _moveDirection *= -1;
84         double dodgeAngle = 85 + _rand.Next(15);
85         SetTurnRight(dodgeAngle - bearing);
86         SetForward((130 + _rand.Next(50)) * _moveDirection);
87     }
88     else
89     {
90         if (e.ScannedBotId == _targetBotId)
91         {
92             SetTurnRight(90 - bearing);
93             SetForward(110 * _moveDirection);
94         }
95     }
96
97     _enemyEnergyHistory[e.ScannedBotId] = e.Energy;
98
99     if (e.ScannedBotId == _targetBotId)
100     {

```

```

100 {
101     double radarBearing = RadarBearingTo(e.X, e.Y);
102     SetTurnRadarLeft(radarBearing * 1.2);
103
104     double firePower = (distance < 200) ? 3.0 : (distance < 500 ? 2.0 : 1.0);
105     if (Energy < 20) firePower = 0.5;
106
107     double bulletSpeed = 20 - 3 * firePower;
108     double travelTime = distance / bulletSpeed;
109
110     double futureX = e.X + Math.Sin(e.Direction * Math.PI / 180.0) * e.Speed * travelTime;
111     double futureY = e.Y + Math.Cos(e.Direction * Math.PI / 180.0) * e.Speed * travelTime;
112
113     double gunBearingToFuture = GunBearingTo(futureX, futureY);
114     SetTurnGunLeft(gunBearingToFuture);
115
116     if (Math.Abs(gunBearingToFuture) < 8 && GunHeat == 0 && Energy > firePower)
117     {
118         SetFire(firePower);
119     }
120 }
121
122 if (distance < 130 && e.ScannedBotId == _targetBotId)
123 {
124     _moveDirection = -1;
125     SetBack(140);
126 }
127 }
128

```

```

129 public override void OnBotDeath(BotDeathEvent e)
130 {
131     _detectedEnemies.Remove(e.VictimId);
132     _enemyEnergyHistory.Remove(e.VictimId);
133     if (_targetBotId == e.VictimId) _targetBotId = -1;
134 }
135
136 public override void OnHitWall(HitWallEvent e)
137 {
138     Stop();
139     double centerBearing = BearingTo(ArenaWidth / 2.0, ArenaHeight / 2.0);
140     Forward(-60 * _moveDirection);
141     TurnLeft(centerBearing);
142     _moveDirection *= -1;
143     Forward(120 * _moveDirection);
144 }
145
146 public override void OnHitBot(HitBotEvent e)
147 {
148     SetFire(3.0);
149     _moveDirection *= -1;
150     SetForward(120 * _moveDirection);
151 }
152 }

```

4.2 Penjelasan Struktur Data, Fungsi, dan Prosedur Bot Terpilih

4.2.1 Struktur Data yang Digunakan

Bot Bokir menggunakan struktur data utama berupa: **Dictionary<int, ScannedBotEvent>**. Struktur data ini digunakan untuk menyimpan seluruh data musuh yang berhasil dideteksi radar. Key berupa BotId, sedangkan value berisi informasi musuh seperti posisi koordinat, arah gerak, energi, dan jarak musuh.

Penggunaan dictionary dipilih karena proses pencarian data musuh menjadi lebih cepat dan efisien. Bot dapat langsung mengambil data berdasarkan ID musuh tanpa perlu melakukan pencarian berulang pada seluruh daftar lawan.

Selain dictionary, bot juga menggunakan beberapa variabel pendukung seperti:

- `_closestEnemyId` untuk menyimpan ID musuh terdekat.
- `_closestDistance` untuk menyimpan jarak musuh terdekat.
- `_inRamMode` untuk menentukan apakah bot sedang berada pada mode tabrakan jarak dekat atau tidak.

Struktur data tersebut membantu bot dalam mengambil keputusan greedy secara cepat pada setiap putaran permainan.

4.2.2 Fungsi-Fungsi Utama

Berikut merupakan beberapa fungsi utama yang digunakan pada implementasi bot Bokir:

1. `Run()`

Fungsi ini merupakan fungsi utama yang dijalankan selama pertandingan berlangsung. Pada fungsi ini bot akan terus melakukan scanning radar, memilih target terdekat, lalu mengejar musuh untuk melakukan serangan.

2. `SelectClosestEnemy()`

Fungsi ini digunakan untuk memilih musuh dengan jarak paling dekat dari seluruh musuh yang terdeteksi radar. Fungsi ini menjadi inti dari algoritma greedy by distance karena bot selalu memilih target yang dianggap paling menguntungkan pada saat itu.

Parameter: tidak ada.
Nilai kembalian: tidak ada.

3. `ChaseAndRam(ScannedBotEvent target)`

Fungsi ini digunakan untuk mengejar musuh dan melakukan tabrakan (ram). Bot akan mengarahkan badan menuju target lalu bergerak maju dengan kecepatan penuh. Jika jarak sudah sangat dekat maka bot akan menembakkan peluru dengan power besar untuk menghasilkan combo damage.

Parameter: `target` → data musuh yang menjadi target utama.
Nilai kembalian: tidak ada.

4. `DistanceTo(x, y)`

Fungsi bawaan Robocode yang digunakan untuk menghitung jarak antara posisi bot dan posisi musuh. Nilai jarak ini digunakan sebagai dasar pengambilan keputusan greedy.

Parameter:

- `x` → koordinat X musuh.
- `y` → koordinat Y musuh.

Nilai kembalian: jarak antara bot dan musuh.

5. **BearingTo(x, y)**

Fungsi untuk menentukan arah atau sudut posisi musuh terhadap bot. Fungsi ini membantu bot menentukan arah gerakan saat mengejar lawan.

Parameter:

- $x \rightarrow$ koordinat X musuh.
- $y \rightarrow$ koordinat Y musuh.

Nilai kembalian: sudut arah target.

6. **GunBearingTo(x, y)**

Fungsi untuk menentukan arah putaran meriam menuju target. Fungsi ini digunakan agar tembakan lebih tepat sasaran.

Parameter:

- $x \rightarrow$ koordinat X musuh.
- $y \rightarrow$ koordinat Y musuh.

Nilai kembalian: sudut arah meriam terhadap target.

4.2.3 Prosedur Event Handler

Bot Bokir menggunakan beberapa event handler untuk merespons kondisi yang terjadi selama pertandingan berlangsung.

1. **OnScannedBot(ScannedBotEvent e)**

Event ini dijalankan ketika radar mendeteksi musuh. Bot akan menyimpan data musuh ke dalam dictionary agar dapat digunakan untuk menentukan target terdekat.

Respons bot:

- Menyimpan posisi dan informasi musuh
- Memperbarui target greedy

2. **OnBotDeath(BotDeathEvent e)**

Event ini dijalankan ketika salah satu musuh hancur. Bot akan menghapus data musuh tersebut dari dictionary agar tidak lagi dipilih sebagai target.

Respons bot:

- Menghapus data musuh yang mati
- Memilih target baru jika diperlukan

3. **OnHitWall(HitWallEvent e)**

Event ini dijalankan ketika bot menabrak dinding arena. Bot akan mundur sebentar lalu memutar arah agar tidak tersangkut di sudut arena.

Respons bot:

- Mundur dari dinding
- Mengubah arah gerakan

4. OnHitBot(HitBotEvent e)

Event ini dijalankan ketika bot bertabrakan dengan musuh. Pada kondisi ini bot langsung menembakkan peluru dengan power maksimum untuk menghasilkan damage tambahan. Respons bot:

- Menembak dengan power besar
- Tetap mendorong musuh untuk menghasilkan ram damage

4.3 Pengujian

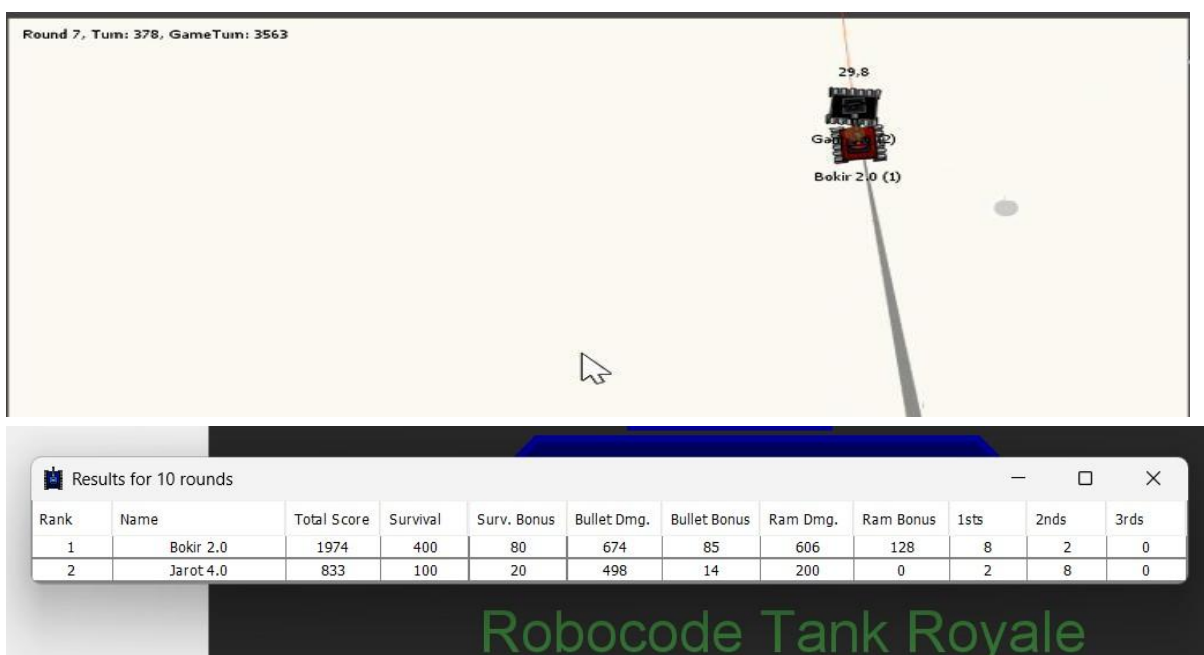
Pengujian dilakukan sebanyak tiga kali dalam mode pertandingan 1 vs 1 menggunakan bot Bokir sebagai bot utama. Setiap pengujian menggunakan skenario yang berbeda untuk melihat performa strategi greedy by distance dalam berbagai kondisi pertandingan.

4.3.1 Pengujian 1: Kondisi Normal pada Arena Standar

Pengujian pertama dilakukan pada arena standar dengan kondisi awal normal. Bot utama Bokir diadu melawan bot Jarot yang menggunakan strategi prediksi arah tembakan. Pada pengujian ini kedua bot memulai pertandingan dengan energi penuh dan posisi awal acak.

Pada awal pertandingan, Bokir langsung bergerak agresif mendekati lawan sesuai heuristic greedy by distance. Jarot mencoba menjaga jarak dan menyerang dari jarak menengah menggunakan prediksi tembakan. Namun Bokir berhasil mendekat dan melakukan beberapa tabrakan yang menghasilkan ram damage cukup besar.

Screenshot/Dokumentasi Pertandingan:



Round 7, Turn: 378, GameTurn: 3563

Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Bokir 2.0	1974	400	80	674	85	606	128	8	2	0
2	Jarot 4.0	833	100	20	498	14	200	0	2	8	0

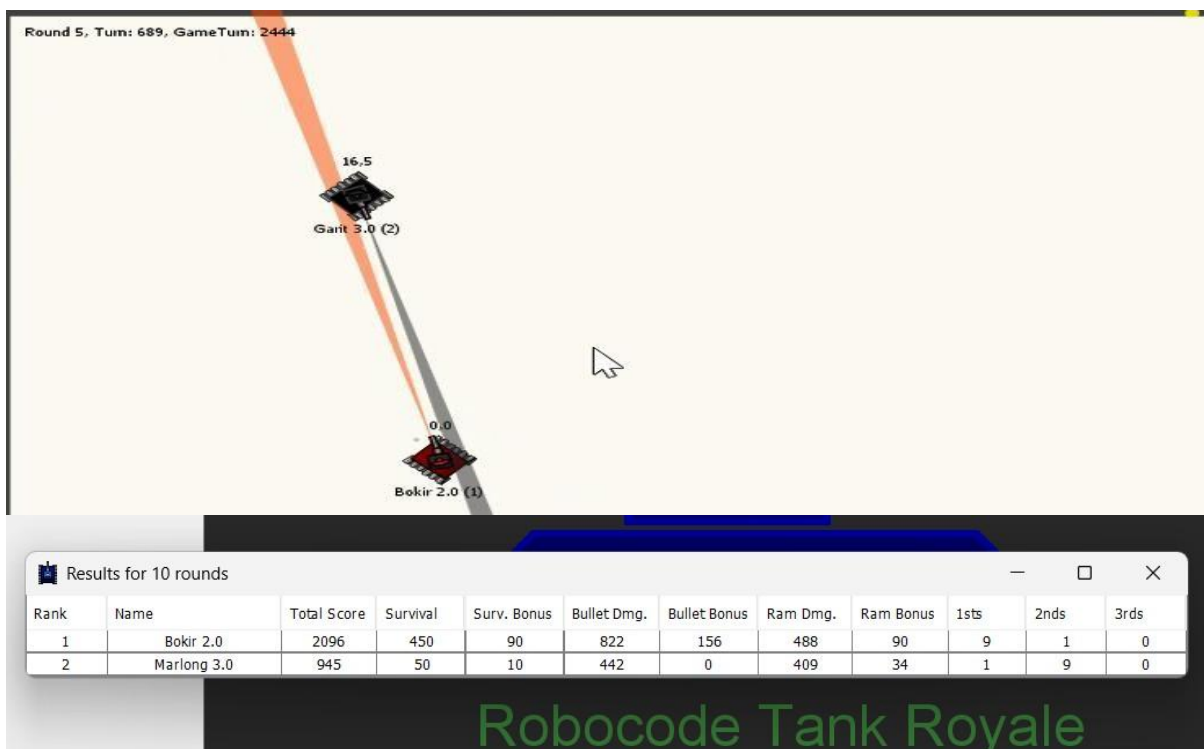
Robocode Tank Royale

Berdasarkan hasil pengujian, strategi greedy by distance bekerja cukup efektif pada arena standar. Bokir mampu terus menekan lawan dengan pendekatan agresif sehingga lawan kesulitan menjaga jarak aman. Kombinasi serangan tabrakan dan tembakan jarak dekat juga memberikan damage yang besar terhadap musuh.

4.3.2 Pengujian 2: Melawan Bot Agresif

Pada pengujian kedua, Bokir diadu melawan Garit yang memiliki pola gerakan agresif dan cukup aktif melakukan serangan balasan. Arena yang digunakan masih arena standar, namun pertandingan berlangsung lebih cepat karena kedua bot sama-sama sering bergerak mendekat. Bokir tetap menggunakan heuristic greedy by distance dengan memilih musuh terdekat lalu mengejar target secara langsung. Dalam beberapa kondisi terjadi tabrakan antar bot yang menghasilkan ram damage cukup tinggi.

Screenshot/Dokumentasi Pertandingan:



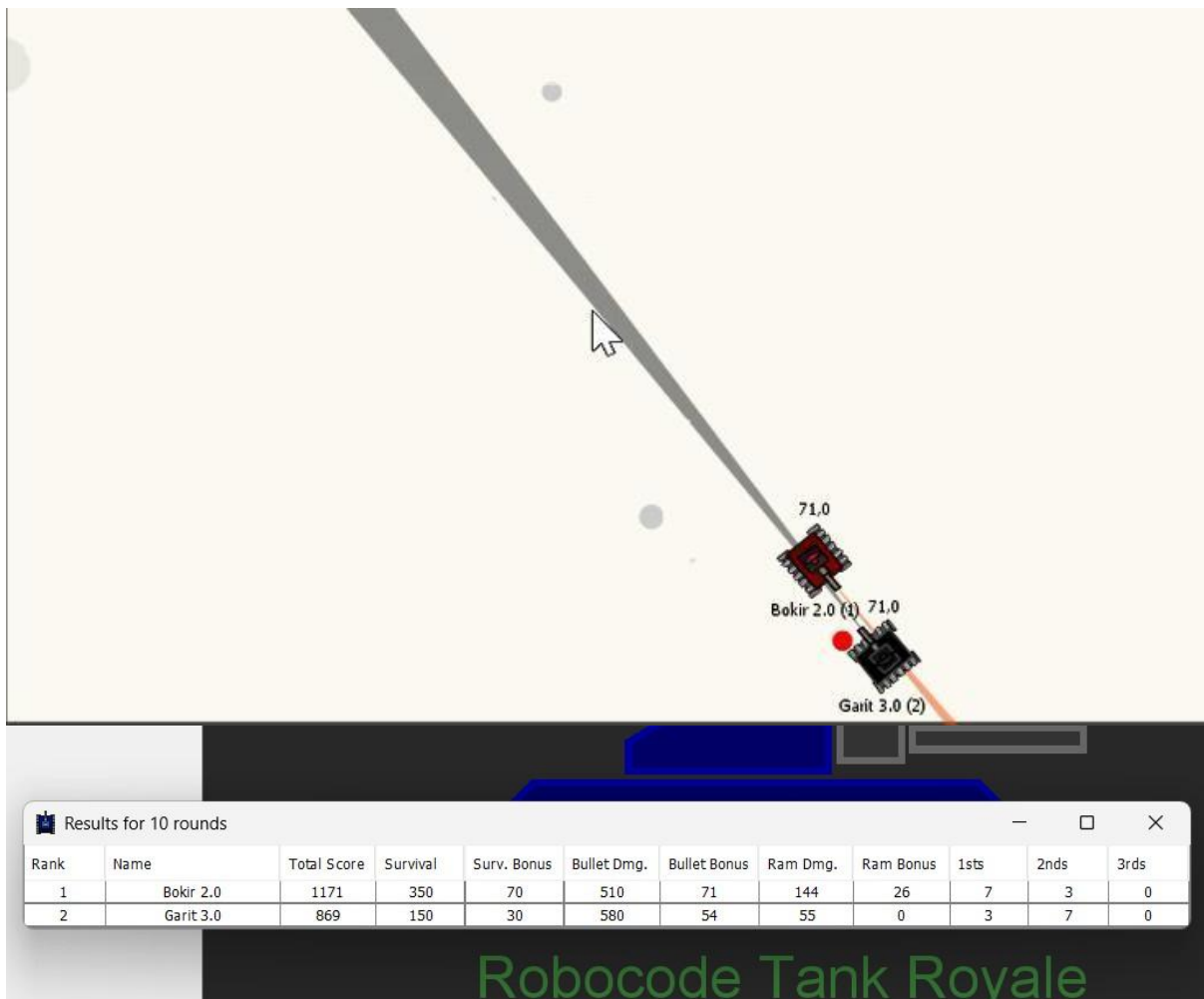
Hasil pengujian menunjukkan bahwa Bokir masih mampu unggul meskipun melawan bot agresif. Strategi greedy membuat Bokir terus memberikan tekanan sehingga lawan tidak memiliki banyak ruang untuk menghindari. Akan tetapi energi Bokir juga cukup cepat berkurang karena sering terjadi kontak langsung dengan lawan.

4.3.3 Pengujian 3: posisi awal terjepit atau bot defensif]

Pada pengujian ketiga, Bokir dihadapkan dengan kondisi posisi awal yang kurang menguntungkan karena berada dekat dinding arena. Lawan yang digunakan adalah Marlong yang memiliki strategi menyerang musuh dengan energi terendah.

Pada awal pertandingan Bokir sempat mengalami kesulitan bergerak karena posisi dekat sudut arena membuat ruang gerakanya terbatas. Namun setelah berhasil keluar dari sudut arena, Bokir kembali menerapkan strategi greedy dengan mengejar target terdekat secara agresif.

Screenshot/Dokumentasi Pertandingan:



Pada pengujian ini terlihat bahwa strategi greedy by distance masih dapat bekerja dengan baik meskipun posisi awal kurang menguntungkan. Namun performa bot sedikit menurun karena Bokir beberapa kali menabrak dinding saat mengejar lawan.

4.4 Analisis Hasil Pengujian

Berdasarkan hasil pengujian yang telah dilakukan, strategi Greedy by Distance yang digunakan pada bot Bokir terbukti cukup efektif dalam pertandingan 1 vs 1. Bot mampu memberikan tekanan secara agresif dengan selalu mengejar musuh terdekat dan melakukan serangan jarak dekat secara terus-menerus.

Strategi ini memberikan keuntungan karena bot dapat lebih cepat mendekati lawan sehingga peluang mengenai target menjadi lebih tinggi. Selain itu, pendekatan ramming yang digunakan Bokir juga mampu menghasilkan Ram Damage dan Ram Damage Bonus secara konsisten.

Pada beberapa kondisi, Bokir mampu memenangkan pertandingan dengan cepat ketika berhasil menjaga jarak dekat dengan lawan. Penggunaan tembakan power tinggi saat berada di dekat target juga meningkatkan total damage yang dihasilkan selama pertandingan.

Namun, strategi greedy ini memiliki beberapa kelemahan. Karena bot terlalu fokus mengejar target terdekat, Bokir terkadang mudah menabrak dinding atau kehilangan posisi yang menguntungkan di arena. Selain itu, saat melawan bot dengan movement yang lincah dan kemampuan dodge yang baik, efektivitas serangan ramming menjadi berkurang.

Secara keseluruhan, strategi Greedy by Distance cocok digunakan pada bot Bokir karena mampu menghasilkan permainan yang agresif, cepat, dan efektif dalam pertarungan jarak dekat. Strategi ini juga memiliki proses pengambilan keputusan yang sederhana sehingga bot dapat merespons kondisi pertandingan dengan cepat pada setiap turn permainan.

BAB 5: KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan implementasi dan pengujian yang telah dilakukan, algoritma greedy dapat diterapkan dengan baik pada permainan Robocode Tank Royale. Setiap bot berhasil mengimplementasikan heuristic greedy yang berbeda sesuai dengan tujuan strateginya masing-masing.

Empat strategi greedy yang berhasil dikembangkan dalam tugas besar ini yaitu:

- Greedy by Distance pada bot Bokir
- Greedy by Enemy Energy pada bot Marlong
- Greedy by Angle Prediction pada bot Jarot
- Greedy by Defensive Movement pada bot Garit

Dari seluruh strategi yang diuji, strategi Greedy by Distance yang diterapkan pada bot Bokir dipilih sebagai strategi utama karena memiliki performa paling sesuai untuk pertandingan 1 vs 1. Strategi ini mampu memberikan tekanan agresif secara konsisten dengan selalu mengejar musuh terdekat dan melakukan serangan jarak dekat untuk memperoleh bullet damage dan ram damage secara maksimal.

Hasil pengujian menunjukkan bahwa Bokir mampu memenangkan pertandingan secara konsisten pada berbagai skenario pengujian. Pengambilan keputusan yang sederhana membuat bot dapat merespons kondisi permainan dengan cepat tanpa memerlukan proses perhitungan yang kompleks.

Melalui tugas besar ini dapat dipahami bahwa algoritma greedy sangat cocok digunakan pada sistem permainan real-time karena memiliki kompleksitas rendah, proses pengambilan keputusan yang cepat, dan implementasi yang relatif sederhana namun tetap efektif.

5.2 Saran

Beberapa saran pengembangan yang dapat dilakukan pada penelitian dan implementasi berikutnya adalah sebagai berikut:

- Menambahkan algoritma machine learning agar bot dapat beradaptasi terhadap pola permainan musuh secara otomatis.
- Mengembangkan sistem prediksi pergerakan musuh yang lebih kompleks untuk meningkatkan akurasi tembakan.
- Mengombinasikan algoritma greedy dengan algoritma lain seperti minimax atau reinforcement learning untuk menghasilkan strategi yang lebih optimal.
- Menambahkan mekanisme penghindaran dinding dan positioning arena yang lebih baik agar bot tidak terlalu mudah kehilangan posisi saat mengejar lawan.
- Mengembangkan strategi kerja sama antar bot pada mode multiplayer sehingga bot dapat saling mendukung dalam pertandingan tim.

LAMPIRAN

Lampiran 1: Tautan Repository GitHub

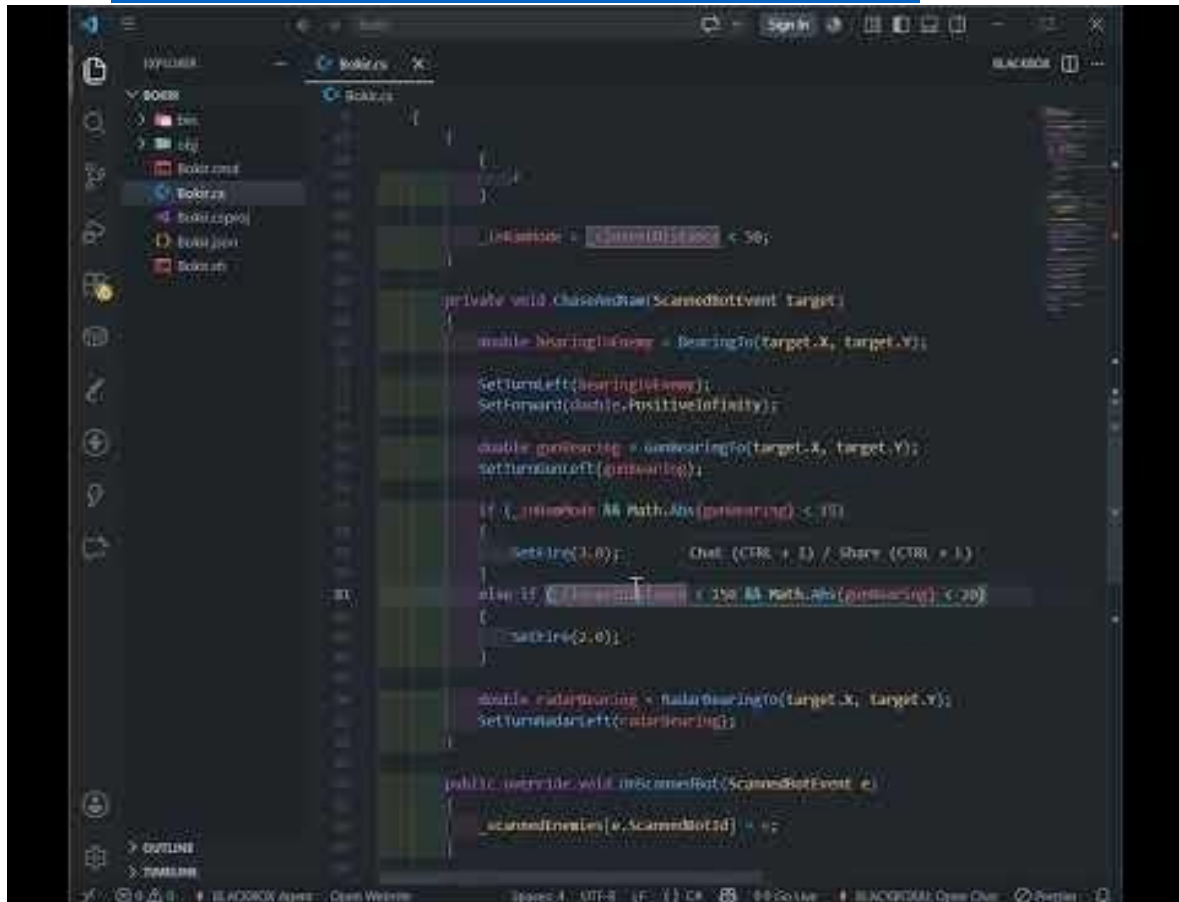
Repository kode sumber bot dapat diakses melalui tautan berikut:

GitHub: https://github.com/KevinSavero/Tubes_Terserah

Lampiran 2: Tautan Video Demo

Video demonstrasi bot dapat diakses melalui tautan berikut:

Video: <https://youtu.be/k7-7Gz70LeI?si=MCOYafrx86525qXO>



Lampiran 3: Tabel Checklist

NO.	Poin	Ya	Tidak
1.	Bot dapat dijalankan pada engine yang sudah dimodifikasi asisten.	✓	
2.	Membuat 4 solusi greedy dengan heuristic yang berbeda	✓	
3.	Membuat laporan sesuai dengan spesifikasi	✓	
4.	Membuat video bonus diunggah pada youtube	✓	

References

- [1] Robocode Tank Royale, "Official Documentation," 2024. [Online]. Available: <https://robocode-dev.github.io/tank-royale/>.
- [2] J. A. and A. H. , "Implementasi Algoritma Greedy pada Pencarian Langkah Optimal Permainan Mahjong Solitaire," *Jurna RESTI (Rekayasa Sistem dan Teknologi Informasi)*, vol. 1, no. 3, pp. 226-231, 2017.
- [3] H. P. and S. S. , "Penerapan Pathfinding Menggunakan Algoritma A* Pada Non Player Character (NPC) Di Game," *Jurnal Ilmiah SINUS*, vol. 17, no. 2, pp. 39-50, 2019.
- [4] M. G. F. J. N. and N. F. , "Implementasi Metode Path Finding dengan Penerapan Algoritma A-Star untuk Mencari Jalur Terpendek pada Game "Jumrah Launch Story"," *Walisongo Journal of Information Technology*, vol. 3, no. 1, pp. 43-48, 2021.
- [5] A. S. J. N. K. A. I. M. and K. R. D. , "A Systematic Review and Analysis of Intelligence-Based Pathfinding Algorithms in the Field of Video Games," *Applied Sciences*, vol. 12, no. 11, p. 5499, 2022.
- [6] Microsoft, "C# Language Documentation," 2024. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/csharp/>.